

Gap Engine Mvp

Gap Engine MVP

Proposito

El primer modulo central del sistema sera el Gap Engine.

Su responsabilidad es comparar el perfil actual de un estudiante contra los requerimientos de un puesto objetivo y generar un informe de brecha accionable.

```
student profile + target role/company requirements
-> Gap Engine
-> gap report JSON
-> Training Module
-> learning path
```

El Gap Engine pertenece a nuestro sistema. Analiza la informacion recibida, calcula la brecha y devuelve un JSON estructurado con el informe.

Este modulo no genera el plan de capacitacion. Solo responde que conocimientos, habilidades o competencias le faltan al estudiante para acercarse al puesto objetivo.

El plan de capacitacion sera responsabilidad de otro modulo interno posterior. Ese modulo consumira el JSON generado por el Gap Engine y lo transformara en modulos, unidades y ejercicios.

Alcance Del MVP

El MVP del Gap Engine debe permitir:

- Recibir datos del estudiante.
- Recibir el puesto objetivo.
- Recibir los requerimientos de habilidades del puesto.
- Comparar nivel actual vs nivel requerido.
- Calcular brecha por habilidad.
- Calcular un score general de preparacion.
- Persistir el informe.
- Consultar el informe generado.

No incluye por ahora:

- IA generativa.
- Cola de mensajeria.

- Procesamiento asincronico.
- Historial avanzado de evaluaciones.
- Modelo complejo de skills en tablas normalizadas.
- Motor semantico de equivalencias entre habilidades.
- Generacion del learning path.

Modulo Propuesto

```
app/modules/gap_analysis/  
  router.py  
  service.py  
  repository.py  
  models.py  
  schemas.py  
  engine.py
```

Responsabilidades:

- `router.py` : endpoints HTTP.
- `service.py` : orquestacion del caso de uso.
- `repository.py` : persistencia.
- `models.py` : tabla principal del informe.
- `schemas.py` : contratos de request/response.
- `engine.py` : algoritmo deterministico de calculo de brecha.

Endpoints MVP

Crear Analisis De Brecha

```
POST /api/v1/gap-analyses
```

Request:

```
{  
  "student": {  
    "name": "Ana Perez",  
    "email": "ana@example.com",  
    "skills": [  
      {  
        "name": "Git",  
        "level": 1  
      },  
      {  
        "name": "SQL",  
        "level": 2  
      }  
    ],  
    "interests": ["logistica", "backend"]  
  },  
  "company": {
```

```

    "name": "Empresa Logistica SA"
  },
  "target_role": {
    "title": "Backend Developer Junior",
    "required_skills": [
      {
        "name": "Git",
        "level": 3,
        "priority": "HIGH"
      },
      {
        "name": "SQL",
        "level": 3,
        "priority": "HIGH"
      },
      {
        "name": "HTTP APIs",
        "level": 2,
        "priority": "MEDIUM"
      }
    ]
  }
}

```

Response:

```

{
  "id": 1,
  "student": {
    "name": "Ana Perez",
    "email": "ana@example.com"
  },
  "company": {
    "name": "Empresa Logistica SA"
  },
  "target_role": {
    "title": "Backend Developer Junior"
  },
  "summary": "Ana Perez necesita reforzar Git, SQL y HTTP APIs para acercarse al puesto",
  "readiness_score": 45,
  "skills": [
    {
      "name": "Git",
      "current_level": 1,
      "required_level": 3,
      "gap_level": 2,
      "priority": "HIGH",
      "status": "MISSING"
    },
    {
      "name": "SQL",
      "current_level": 2,
      "required_level": 3,
      "gap_level": 1,
      "priority": "HIGH",
      "status": "NEEDS_WORK"
    },
    {

```

```
    "name": "HTTP APIs",
    "current_level": 0,
    "required_level": 2,
    "gap_level": 2,
    "priority": "MEDIUM",
    "status": "MISSING"
  }
],
"created_at": "2026-05-12T15:30:00"
}
```

Consultar Analisis

```
GET /api/v1/gap-analyses/{id}
```

Devuelve el informe previamente generado.

Reglas De Calculo

Nivel De Habilidad

Para el MVP, cada habilidad se representa con un nivel numerico simple:

```
0 = no declarado / no sabe
1 = basico
2 = inicial operativo
3 = requerido para junior
4 = intermedio
5 = avanzado
```

Esta escala puede cambiar, pero es suficiente para validar el flujo.

Gap Por Habilidad

```
gap_level = required_level - current_level
```

Si el estudiante no declara una habilidad requerida, se asume:

```
current_level = 0
```

Estado Por Habilidad

```
READY      -> gap_level <= 0
NEEDS_WORK -> gap_level == 1
MISSING     -> gap_level >= 2
```

Prioridad

Prioridades aceptadas:

```
HIGH  
MEDIUM  
LOW
```

Pesos iniciales:

```
HIGH    = 3  
MEDIUM = 2  
LOW     = 1
```

Readiness Score

El score mide que tan cerca esta el estudiante del puesto objetivo.

Formula MVP:

```
skill_score = min(current_level, required_level) / required_level  
weighted_score = skill_score * priority_weight  
readiness_score = sum(weighted_score) / sum(priority_weight) * 100
```

Resultado:

```
0    = no cubre ningun requerimiento  
100 = cubre todos los requerimientos
```

Persistencia MVP

Tabla:

```
gap_analyses
```

Campos:

```
id  
student_name  
student_email  
company_name  
target_role_title  
readiness_score  
summary  
input_json  
result_json  
created_at  
updated_at
```

Para el MVP se guardan `input_json` y `result_json` como JSON.

Motivo: evita modelar prematuramente tablas de skills, roles, empresas, evaluaciones y equivalencias.
Cuando el flujo este validado, se puede normalizar.

Ejemplo De Interpretacion

Si el puesto requiere:

```
Git nivel 3 HIGH
SQL nivel 3 HIGH
HTTP APIs nivel 2 MEDIUM
```

Y el estudiante tiene:

```
Git nivel 1
SQL nivel 2
HTTP APIs no declarado
```

Entonces:

```
Git      -> gap 2 -> MISSING
SQL      -> gap 1 -> NEEDS_WORK
HTTP APIs -> gap 2 -> MISSING
```

El informe debería indicar que Git y HTTP APIs son las brechas principales, y SQL necesita refuerzo.

Relacion Con El Modulo De Capacitacion

El Gap Engine produce el insumo principal para el modulo de capacitacion.

Ambos modulos son parte del mismo backend modular:

```
app/modules/gap_analysis/
    -> genera GapReport JSON

app/modules/learning_paths/
    -> consume GapReport JSON
    -> genera plan de capacitacion
```

Flujo posterior:

```
GapAnalysis.result_json
    -> Learning Path Generator
    -> Training Plan
```

El modulo de capacitacion tomara:

- habilidades faltantes,

- prioridad,
- nivel actual,
- nivel requerido,
- intereses del estudiante,
- puesto objetivo,
- empresa,

y generara:

- modulos,
- unidades,
- ejercicios,
- orden sugerido,
- tutor asignado,
- criterios de avance.

Decisiones De Simplicidad

Para maximizar time to market:

- El calculo sera deterministico.
- No habra IA en el primer corte.
- No habra procesamiento asincronico.
- No se usara cola.
- No se normalizaran habilidades todavia.
- Se persistira input y output completo para trazabilidad.
- No se implementara login en este primer corte tecnico.

La autenticacion queda diferida para antes de presentar el MVP. Mientras tanto, los endpoints se consideran publicos solo para desarrollo local o demo controlada.

Antes de presentar el MVP se debera agregar:

- registro/login basico,
- JWT,
- proteccion de endpoints,
- roles minimos para estudiante, tutor, empresa y admin.

Esto permite demostrar rapido el valor central:

dados un estudiante y un puesto objetivo,
el sistema puede explicar claramente la brecha.

Proximo Paso De Implementacion

Implementar:

```
POST /api/v1/gap-analyses
GET /api/v1/gap-analyses/{id}
```

con:

- validacion Pydantic,
- calculo en `engine.py`,
- persistencia en SQLite local/PostgreSQL Render via SQLAlchemy,
- respuesta JSON lista para ser consumida por el futuro modulo de capacitacion.